RICHFIELD GRADUATE INSTITUTE OF TECHNOLOGY

PROGRAMMING 742

ASSIGNMENT REPORT

Staybnb - Airbnb-Inspired Managed Property Rental Ecosystem

Name and surname	[Enter student name]
Student number	[Enter student number]
Module	Programming 742
Submission date	13 September 2026

ASSIGNMENT COMPLETION CHECK

[X] Times New Roman, size 12 and 1.5 line spacing used.

[X] Harvard-style citations and references included.

[X] Table of Contents, Introduction, Main Body and Conclusion included.

[X] Complete source code, migration, tests and supporting evidence included.

[X] Report supplied in PDF format.

DECLARATION OF ORIGINALITY

I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgement is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism.

Student signature: _____ Date: _____

TABLE OF CONTENTS

1. Introduction 3

2. System Design 3

3. Data Design 4

4. Identity and Role Evolution 5

5. Hosting Workflow 6

6. Guest Experience 7

7. Administration, Communication and Logging 8

8. Validation and Security 9

9. Testing and Verification 10

10. Requirements-to-Implementation Summary 11

11. Conclusion 12

References 12

Appendix A: Demonstration Accounts 13

1. INTRODUCTION

Staybnb is an ASP.NET Core MVC web application that demonstrates the user lifecycle described in the Programming 742 assignment brief. A person registers as a Guest, explores active properties, requests a booking and completes check-in. A Guest can also submit a property application. An Admin reviews that application, and approval changes the account from Guest to Host. The Host then manages listings, booking requests and guest documents. A SuperAdmin oversees role promotion and the system activity record.

The project uses ASP.NET Core Identity because it provides standard support for accounts, stored passwords and roles (Microsoft, 2026a). The work deliberately uses a shallow design that a student group can explain: a Razor page sends a request to an MVC controller, the controller reads or writes through one Entity Framework Core database context, and SQL Server stores the result.

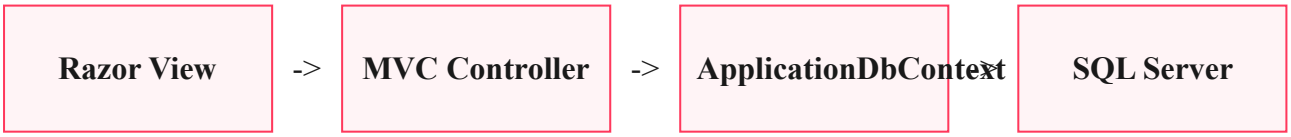
1.1 Project objectives

- Implement authentication and four clear roles: Guest, Host, Admin and SuperAdmin.
- Demonstrate the Guest-to-Host change while ensuring the two roles are not held together.
- Provide a complete property application, booking and document-verification workflow.
- Store messages, notifications and important activity in SQL Server.
- Keep the code readable, directly connected to the rubric and simple to demonstrate.

2. SYSTEM DESIGN

2.1 Chosen structure

The application is one MVC project supported by one small test project. It does not use a separate API service, repository layer or message broker because those additions are not required to demonstrate the assessed concepts.



2.2 Main folders

Controllers hold page actions and role checks. Models store information, ViewModels define forms, Data configures the database, Views hold pages, and Services contain small shared operations.

2.3 Access design

Each role-specific controller uses an explicit Authorize role rule, so the server checks access instead of relying on hidden menu items (Microsoft, 2026b).

3. DATA DESIGN

The central database relationships are summarised below. Primary keys identify rows and foreign keys connect each workflow to the correct account or property.

Entity	Purpose and main relationships
ApplicationUser	Identity account connected to roles, properties, bookings, messages and notifications.
HostProperty	Listing owned by one user. Stores rates, address, type, active status and check-in requirements.
HostApplication	Connects an applicant to one proposed property and records Pending, Approved or Rejected.
PropertyImage	Stores one or more image paths. An application cannot be submitted without an image.
Booking	Connects a Guest to a property and stores dates, travellers, total and workflow status.
GuestDocument	Connects a private ID or passport upload to an approved booking and Host verification.
Message / Notification	Stores inbox conversations and user status updates with Read or Unread state.
ActivityLog	Records the user, action, detail, time and optional network address.
Payment / Review / Amenity / WishlistItem	Supporting entities shown in the supplied relationship guidance.

3.1 Entity Framework Core and SQL Server

ApplicationDbContext maps the entity classes to SQL Server tables, specifies decimal precision for money and restricts selected deletion paths. The Microsoft SQL Server provider is the official Entity Framework Core provider for SQL Server (Microsoft, 2026c).

A complete InitialCreate migration is included. Migrations keep the database structure aligned with the application's data model and record applied changes in a history table (Microsoft, 2026d). The migration was applied successfully to StaybnbDb during verification.

4. IDENTITY AND ROLE EVOLUTION

4.1 Registration and login

Registration collects a first name, last name, email address, password and confirmation. Identity hashes the password before storage. After account creation the controller explicitly adds only the Guest role. Successful login reads the role and opens the matching dashboard. Each successful login also creates an ActivityLog row.

4.2 Startup roles and SuperAdmin

RoleSeeder runs during application startup. It creates Guest, Host, Admin and SuperAdmin when they do not exist, then ensures the demonstration SuperAdmin is present. The Admin-promotion action has its own SuperAdmin role rule. An ordinary Admin can review applications but cannot grant Admin access.

4.3 Guest-to-Host transition

When an Admin approves a first Host application, the program adds Host, removes Guest, marks the application Approved and activates the property. It also creates a notification and audit entries for the role change and application decision.

> **Verified transition**
>
> The seeded applicant began as a Guest with an inactive Durban property. After Admin approval, a fresh login opened the Host dashboard, the Guest dashboard returned Access Denied and the Durban property appeared in public discovery.

4.4 Strict role separation

GuestDashboardController accepts Guest only. HostDashboardController accepts Host only. AdminController accepts Admin or SuperAdmin, while PromoteToAdmin accepts SuperAdmin only. Messages and Notifications accept any signed-in account but filter every row by the current user.

5. HOSTING WORKFLOW

5.1 Property application

A Guest or existing Host supplies a title, description, nightly price, cleaning fee, service fee, address, city, property type and image. Both form validation and an explicit server check reject a missing image. The proposed property is inactive while the HostApplication remains Pending.

5.2 Listing management

The Host dashboard lists only properties belonging to the signed-in Host. The Host can read and edit details, toggle IsActive, configure check-in rules and delete a property that has no booking history. A property with booking history must be deactivated so that its past records remain understandable.

5.3 Booking decisions

The Host sees Pending bookings for owned properties and may approve or reject each request. The server checks ownership and current status before making the change. The Guest receives a notification and the ActivityLog records the booking status change.

5.4 Document verification

An approved booking enables Guest check-in. When a Guest uploads the required document, the Host receives a notification. Only the Host owning that booking's property can download the private file. Verification changes the document to Verified and the booking to CheckedIn; rejection allows a replacement upload.

Host task	Controller action	Protection
Edit property	EditProperty	Host role and HostId match
Toggle listing	ToggleActive	Host role and HostId match
Approve booking	BookingDecision	Host owns property; status is Pending
Verify document	VerifyDocument	Host owns property; document is Pending

6. GUEST EXPERIENCE

6.1 Property discovery

The public home page shows only active properties in a responsive gallery. City text and property type filters are applied in the SQL query. Each card links to a detail page with the host, address and fee information needed before a booking request.

6.2 Booking calculation

The Guest chooses check-in and check-out dates and the number of travellers. Past check-in dates and invalid date order are rejected. The controller loads current prices from SQL Server and applies this formula:

TotalPrice = (PricePerNight x Nights) + CleaningFee + ServiceFee

The browser does not submit a trusted total. This prevents a user from changing the price with developer tools. A verified example used three nights at R1,250 plus R250 cleaning and R150 service fees, producing R4,150.

6.3 Multi-step check-in

The check-in page presents three understandable stages:

1. The Host has approved the booking.
2. The Guest uploads the property's required ID or passport.
3. The Host verifies the document and the booking becomes CheckedIn.

Only the Guest who owns an Approved booking may upload. The property decides which document type is required. A verified or still-pending document blocks duplicate uploads, while a rejected document may be replaced.

6.4 Guest dashboard

The dashboard lists booking dates, travellers, total price and current status. The check-in button appears only for Approved or CheckedIn bookings, making the workflow visible without introducing a complex state framework.

7. ADMINISTRATION, COMMUNICATION AND LOGGING

7.1 Administrative oversight

The Admin dashboard combines the three required oversight tasks. Pending Host and property applications can be approved or rejected. Recent activity is displayed newest first. If the account is SuperAdmin, a list of Guests and promotion controls is also displayed.

7.2 Messages

Messages are normal SQL Server records, so no real-time service is required. A signed-in user can compose using the recipient's email, view sent and received messages, open an item and reply. Opening an unread received message sets IsRead to true. Sending a message creates a New Message notification.

7.3 Notifications

The notification centre displays role, application, booking, check-in and message events. A user can open one notification or mark all as read. Every query includes the current UserId, preventing one account from changing another account's notifications.

7.4 Activity logging

The ActivityLog records login, logout, registration, role changes, Host applications, property changes, booking creation, booking status changes and document decisions. Entries include a timestamp, meaningful description and acting user where available.

Event	Example logged action
Successful login	Login
Guest becomes Host or Admin	RoleChanged
Host approves or rejects a booking	BookingStatusChanged
Host verifies a check-in document	BookingStatusChanged with Approved-to-CheckedIn detail

8. VALIDATION AND SECURITY

The project validates at the page boundary and repeats important checks on the server. The main controls are:

- Anti-forgery validation on every form that changes data.
- Role authorization on every role-specific controller and the SuperAdmin promotion action.
- Ownership checks for bookings, properties, messages, notifications and documents.
- Server-side price calculation using current database values.
- Five-megabyte upload limits, approved extensions and random stored filenames.
- Private check-in storage under App_Data rather than the public web folder.
- Required values, maximum lengths, foreign keys and money precision in the database model.

Microsoft recommends dedicated upload storage, safe application-generated names, approved extensions, server-side checks and size limits (Microsoft, 2026e). The image and check-in upload code follows these principles at a level suitable for this assignment.

8.1 Intentional student-project limits

The system does not add cloud storage, live chat, payment-gateway integration or distributed services. These would make the demonstration harder without earning marks in the supplied rubric. The Payment entity is present because it appears in the supplied relationship guidance, but no false payment flow is presented as complete.

8.2 Production extensions

A production deployment should add malware scanning for uploads, secrets outside configuration files, verified email delivery, account recovery, payment-provider callbacks and a durable external file store. These are documented as extensions rather than hidden behind unnecessary abstractions.

9. TESTING AND VERIFICATION

Check	Result	Evidence
Solution build	Passed	Both projects compiled with 0 warnings and 0 errors.
Focused tests	7 of 7 passed	Three price cases, missing-image validation and three property-activation cases.
Initial migration	Passed	StaybnbDb tables, keys, indexes and migration history created on SQL Server LocalDB.
Public pages	Passed	Home and Login returned HTTP 200; the home contained seeded listings.
Signed-out protection	Passed	GuestDashboard redirected to Account/Login.
Role separation	Passed	Each role reached its allowed dashboard; mismatched dashboards ended at Access Denied.
Role transition	Passed	Applicant became Host, lost Guest access and the approved property became active.
Booking workflow	Passed	Booking #3 stored R4,150, changed Pending to Approved and opened check-in.
Document workflow	Passed	PDF uploaded privately, appeared for its Host and verification produced CheckedIn.

9.1 Focused automated tests

The xUnit project tests the central price formula with ordinary dates, time-of-day values and an invalid zero-night stay. It checks that a Host application without an image is invalid and verifies that submitted properties cannot be activated before approval. These seven tests focus on assessed behaviour and remain short enough for a student to explain.

9.2 Verification boundary

The application was compiled, tested, migrated and exercised through live HTTP requests against SQL Server LocalDB. No in-app or connected external browser was available. The responsive HTML and CSS were inspected in source, but a final manual browser pass on the submission computer remains advisable.

10. REQUIREMENTS-TO-IMPLEMENTATION SUMMARY

Rubric area	Implementation evidence
Authentication and roles - 20	Identity registration/login, Guest default, RoleSeeder, approved Guest-to-Host swap and explicit dashboard role rules.
Admin dashboard - 15	Pending application review, newest-first activity log and SuperAdmin-only Admin promotion.
Host dashboard - 15	Listing management, approval-protected active toggle, booking decisions, check-in setup and document verification.
Application process - 15	Required property information and image, Pending property state and Admin decision.
Booking and check-in - 15	Server price formula and Pending to Approved to CheckedIn workflow.
Messaging and notifications - 10	SQL-backed inbox, replies, notification centre and stored Read/Unread status.
Data and architecture - 10	SQL Server provider, complete migration, clear relationships, ActivityLog and readable MVC structure.

10.1 Requirement traceability in the submission

The source folder also contains docs/REQUIREMENTS-TRACEABILITY.md. It gives a direct path from each requirement to controller, model or test evidence. README.md gives a marking walkthrough and all demonstration accounts.

10.2 Main implementation locations

- AccountController: registration, login and Guest default.
- RoleSeeder: startup roles, SuperAdmin and demonstration data.
- AdminController: application decisions and Admin promotion.
- HostDashboardController: property, booking and document management.
- GuestDashboardController: booking and multi-step check-in.
- MessagesController and NotificationsController: communication state.
- ApplicationDbContext and Migrations: SQL Server structure.

11. CONCLUSION

Staybnb satisfies the assessed user lifecycle with a direct and explainable ASP.NET Core MVC solution. New users begin as Guests; a reviewed property application changes a Guest into a Host; Host and Guest dashboards remain separate; and SuperAdmin retains the only Admin-promotion path. The booking formula, document-based check-in, inbox, notification centre and activity log are stored in SQL Server and connected through clear relationships.

The design avoids unnecessary technologies and layers while still validating important input, protecting owned data and testing the core rules. The included migration, startup data, demonstration accounts, test project and traceability document make the project straightforward for a marker to run and evaluate.

REFERENCES

Microsoft (2026a) 'Introduction to Identity on ASP.NET Core'. Available at: https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity?view=aspnetcore-8.0 (Accessed: 13 September 2026).

Microsoft (2026b) 'Role-based authorization in ASP.NET Core MVC'. Available at: https://learn.microsoft.com/en-us/aspnet/core/mvc/security/authorization/roles?view=aspnetcore-8.0 (Accessed: 13 September 2026).

Microsoft (2026c) 'Microsoft SQL Server database provider - EF Core'. Available at: https://learn.microsoft.com/en-us/ef/core/providers/sql-server/ (Accessed: 13 September 2026).

Microsoft (2026d) 'Migrations overview - EF Core'. Available at: https://learn.microsoft.com/en-us/ef/core/managing-schemas/migrations/ (Accessed: 13 September 2026).

Microsoft (2026e) 'Upload files in ASP.NET Core'. Available at: https://learn.microsoft.com/en-us/aspnet/core/mvc/models/file-uploads?view=aspnetcore-8.0 (Accessed: 13 September 2026).

Richfield Graduate Institute of Technology (2026) 'Programming 742 Assignment: Staybnb - Airbnb Clone'. Unpublished assignment brief.

APPENDIX A: DEMONSTRATION ACCOUNTS

Role	Email	Password
SuperAdmin	superadmin@staybnb.local	Admin123!
Admin	admin@staybnb.local	Admin123!
Host	host@staybnb.local	Host123!
Guest	guest@staybnb.local	Guest123!
Pending applicant	applicant@staybnb.local	Guest123!

These are local demonstration credentials created programmatically on first startup. They are included only so the marking workflow can be repeated quickly.

Suggested marking walkthrough

1. Browse active homes and use city or property-type filtering.
2. Log in as Guest and request a booking.
3. Log in as Host and approve the booking.
4. Return as Guest and upload the required check-in document.
5. Return as Host and verify the document.
6. Log in as Admin and review the pending property application.
7. Log in as SuperAdmin to inspect audit logs and the Admin-promotion control.
8. Open Messages and Notifications to demonstrate stored Read/Unread changes.

Commands

Run: dotnet tool restore; dotnet restore Staybnb.sln; dotnet run --project Staybnb.Web

Test: dotnet test Staybnb.sln

> **Before submission**
>
> Replace the student-name and student-number placeholders on the cover page and sign the declaration of originality.